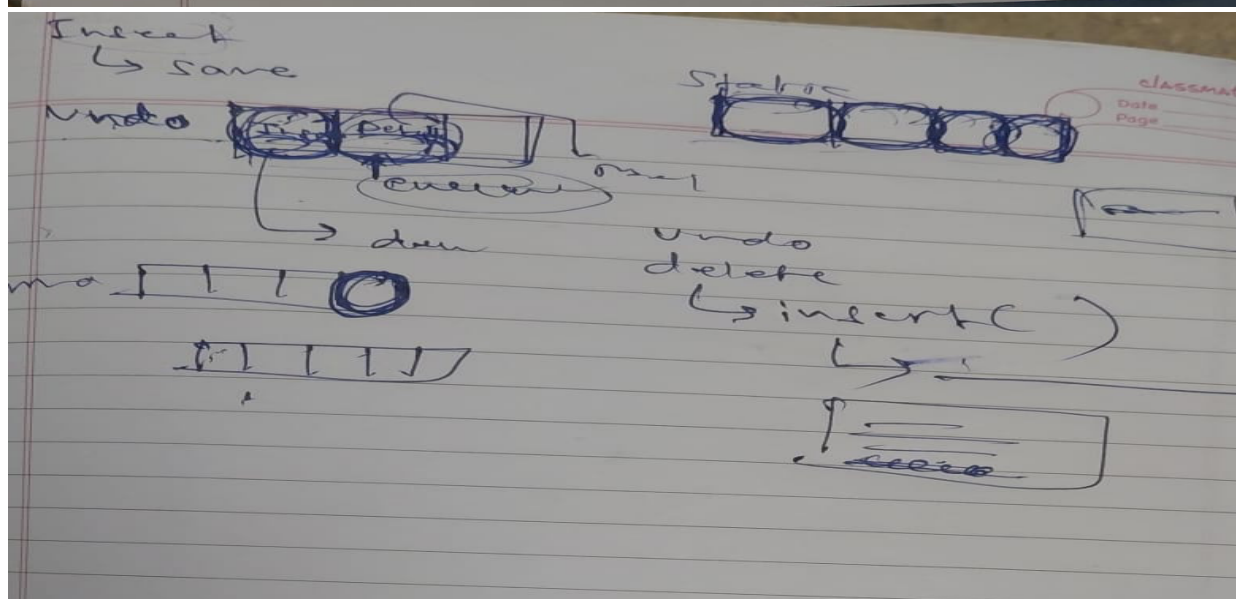
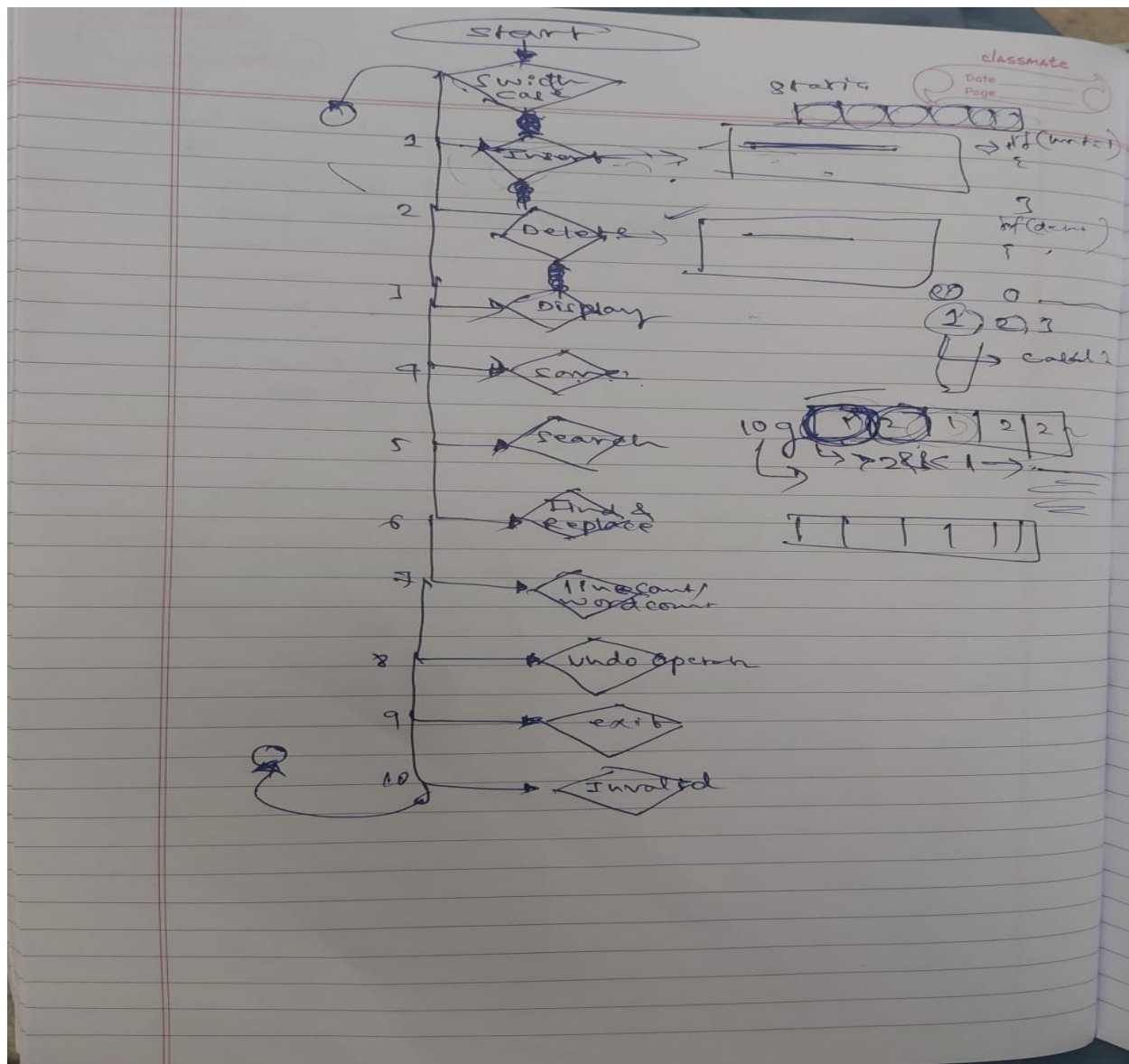


1. PAPER DESIGN



2.WORKING C CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LINES 100
#define MAX_LEN 256

char lines[MAX_LINES][MAX_LEN];
int line_count = 0;

// Undo tracking variables
int last_action = 0;          // 0: None, 1: Insert was done, 2: Delete was done
int undo_line_num = 0;
char undo_text[MAX_LEN];

void print_help() {
    printf("\n--- Advanced Line Editor Help ---\n");
    printf("  p                - Display document state with line numbers\n");
    printf("  i <line> <text>    - Insert a line at a given number\n");
    printf("  d <line>          - Delete a line at a given number\n");
    printf("  u                - Undo last insert or delete action (Bonus!)\n");
    printf("  f <keyword>       - Search for a word or phrase (Bonus!)\n");
    printf("  s <filename>      - Save document to text file\n");
    printf("  l <filename>      - Load document from text file\n");
    printf("  h                - Show this help menu\n");
    printf("  q                - Quit the editor\n");
    printf("-----\n");
}

void display_document() {
    if (line_count == 0) {
        printf("Document is empty.\n");
        return;
    }
    printf("\n--- Document State ---\n");
    for (int i = 0; i < line_count; i++) {
        printf("%3d: %s", i + 1, lines[i]);
    }
    printf("-----\n");
}

void insert_line(int line_num, const char *text, int record_undo) {
    if (line_num < 1 || line_num > line_count + 1) {
        printf("Error: Invalid line number.\n");
        return;
    }
    if (line_count >= MAX_LINES) {
        printf("Error: Document is full.\n");
        return;
    }
}
```

```

}

// Shift lines down
for (int i = line_count; i >= line_num; i--) {
    strcpy(lines[i], lines[i - 1]);
}

strcpy(lines[line_num - 1], text);
if (strchr(lines[line_num - 1], '\n') == NULL) {
    strcat(lines[line_num - 1], "\n");
}

line_count++;

if (record_undo) {
    last_action = 1; // 1 means last action was an insert
    undo_line_num = line_num;
}
printf("Line inserted successfully.\n");
}

void delete_line(int line_num, int record_undo) {
    if (line_num < 1 || line_num > line_count) {
        printf("Error: Invalid line number.\n");
        return;
    }

    if (record_undo) {
        last_action = 2; // 2 means last action was a delete
        undo_line_num = line_num;
        strcpy(undo_text, lines[line_num - 1]);
    }

    // Shift lines up
    for (int i = line_num - 1; i < line_count - 1; i++) {
        strcpy(lines[i], lines[i + 1]);
    }

    line_count--;
    printf("Line deleted successfully.\n");
}

void undo_last_action() {
    if (last_action == 0) {
        printf("Error: Nothing to undo.\n");
        return;
    }

    if (last_action == 1) {
        // Reverse of insert is deleting that line (without re-recording undo)
        delete_line(undo_line_num, 0);
    }
}

```

```

        printf("Undo successful: Reversed last insert.\n");
    } else if (last_action == 2) {
        // Reverse of delete is inserting that line back (without re-recording undo)
        insert_line(undo_line_num, undo_text, 0);
        printf("Undo successful: Restored deleted line.\n");
    }
    last_action = 0; // Reset after undo
}

void search_keyword(const char *keyword) {
    int found = 0;
    printf("\n--- Search Results for '%s' ---\n", keyword);
    for (int i = 0; i < line_count; i++) {
        if (strstr(lines[i], keyword) != NULL) {
            printf("Line %d: %s", i + 1, lines[i]);
            found = 1;
        }
    }
    if (!found) {
        printf("Keyword not found in the document.\n");
    }
    printf("-----\n");
}

void save_file(const char *filename) {
    FILE *file = fopen(filename, "w");
    if (!file) {
        printf("Error: Could not open file for writing.\n");
        return;
    }
    for (int i = 0; i < line_count; i++) {
        fputs(lines[i], file);
    }
    fclose(file);
    printf("Document saved to %s successfully.\n", filename);
}

void load_file(const char *filename) {
    FILE *file = fopen(filename, "r");
    if (!file) {
        printf("Error: Could not open file for reading.\n");
        return;
    }
    line_count = 0;
    last_action = 0;
    while (fgets(lines[line_count], MAX_LEN, file) != NULL && line_count < MAX_LINES) {
        line_count++;
    }
    fclose(file);
    printf("Document loaded from %s successfully.\n", filename);
}

```

```

int main() {
    char cmd[10];
    char filename[50];
    char text[MAX_LEN];
    int line_num;

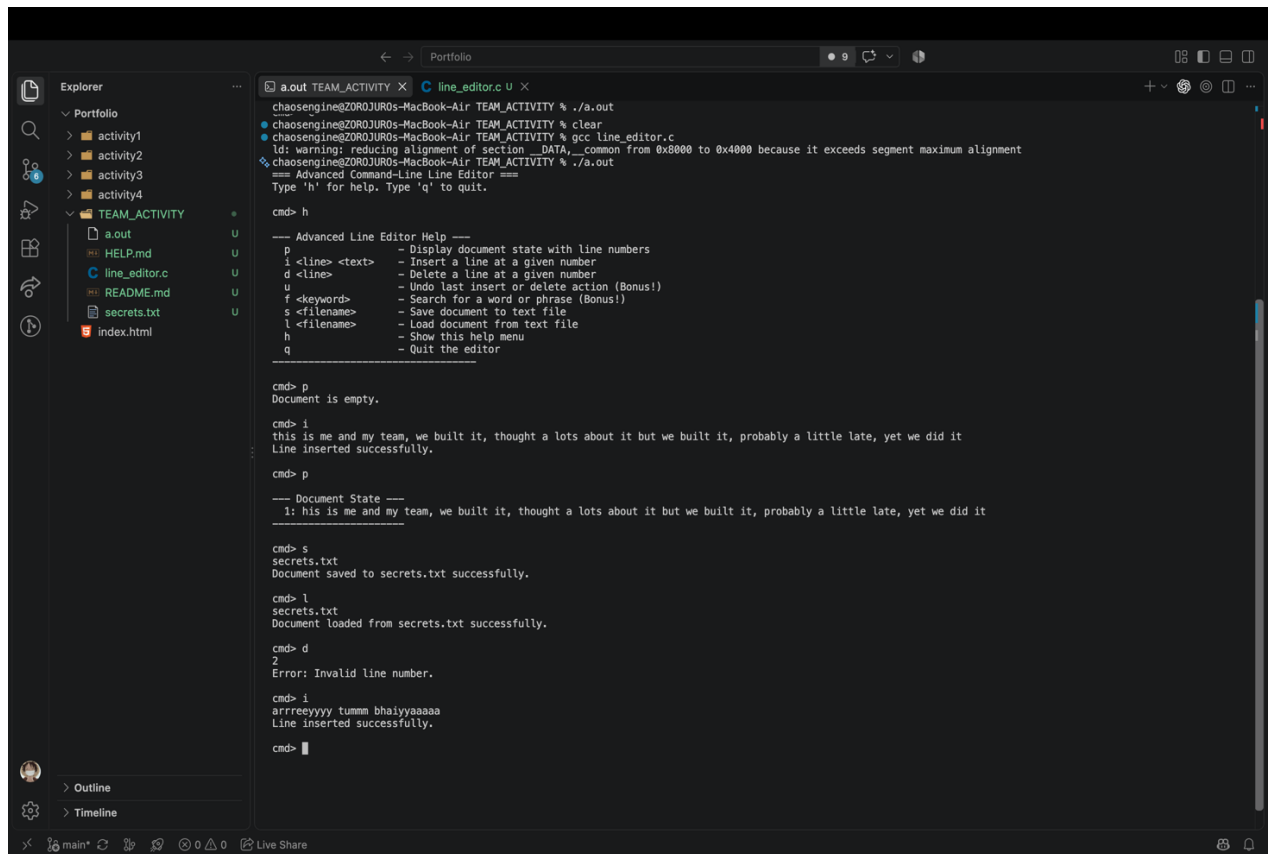
    printf("=== Advanced Command-Line Line Editor ===\n");
    printf("Type 'h' for help. Type 'q' to quit.\n");

    while (1) {
        printf("\ncmd> ");
        if (scanf("%s", cmd) != 1) break;

        if (strcmp(cmd, "q") == 0) {
            break;
        } else if (strcmp(cmd, "h") == 0) {
            print_help();
        } else if (strcmp(cmd, "p") == 0) {
            display_document();
        } else if (strcmp(cmd, "i") == 0) {
            scanf("%d", &line_num);
            getchar(); // clear newline buffer
            fgets(text, MAX_LEN, stdin);
            insert_line(line_num, text, 1);
        } else if (strcmp(cmd, "d") == 0) {
            scanf("%d", &line_num);
            delete_line(line_num, 1);
        } else if (strcmp(cmd, "u") == 0) {
            undo_last_action();
        } else if (strcmp(cmd, "f") == 0) {
            scanf("%s", text);
            search_keyword(text);
        } else if (strcmp(cmd, "s") == 0) {
            scanf("%s", filename);
            save_file(filename);
        } else if (strcmp(cmd, "l") == 0) {
            scanf("%s", filename);
            load_file(filename);
        } else {
            printf("Unknown command. Type 'h' for help.\n");
        }
    }
    return 0;
}

```

3.OUTPUT OF THE CODE COMPILED USING GCC



```
chaosengine@ZOR0JUR0s-MacBook-Air TEAM_ACTIVITY % ./a.out
chaosengine@ZOR0JUR0s-MacBook-Air TEAM_ACTIVITY % clear
chaosengine@ZOR0JUR0s-MacBook-Air TEAM_ACTIVITY % gcc line_editor.c
ld: warning: reducing alignment of section __DATA,__common from 0x8000 to 0x4000 because it exceeds segment maximum alignment
chaosengine@ZOR0JUR0s-MacBook-Air TEAM_ACTIVITY % ./a.out
=== Advanced Command-Line Line Editor ===
Type 'h' for help. Type 'q' to quit.

cmd> h

--- Advanced Line Editor Help ---
p          - Display document state with line numbers
i <line> <text> - Insert a line at a given number
d <line>      - Delete a line at a given number
u          - Undo last insert or delete action (Bonus!)
f <keyword>   - Search for a word or phrase (Bonus!)
s <filename>  - Save document to text file
l <filename>  - Load document from text file
h          - Show this help menu
q          - Quit the editor

cmd> p
Document is empty.

cmd> i
this is me and my team, we built it, thought a lots about it but we built it, probably a little late, yet we did it
Line inserted successfully.

cmd> p

--- Document State ---
1: his is me and my team, we built it, thought a lots about it but we built it, probably a little late, yet we did it

cmd> s
secrets.txt
Document saved to secrets.txt successfully.

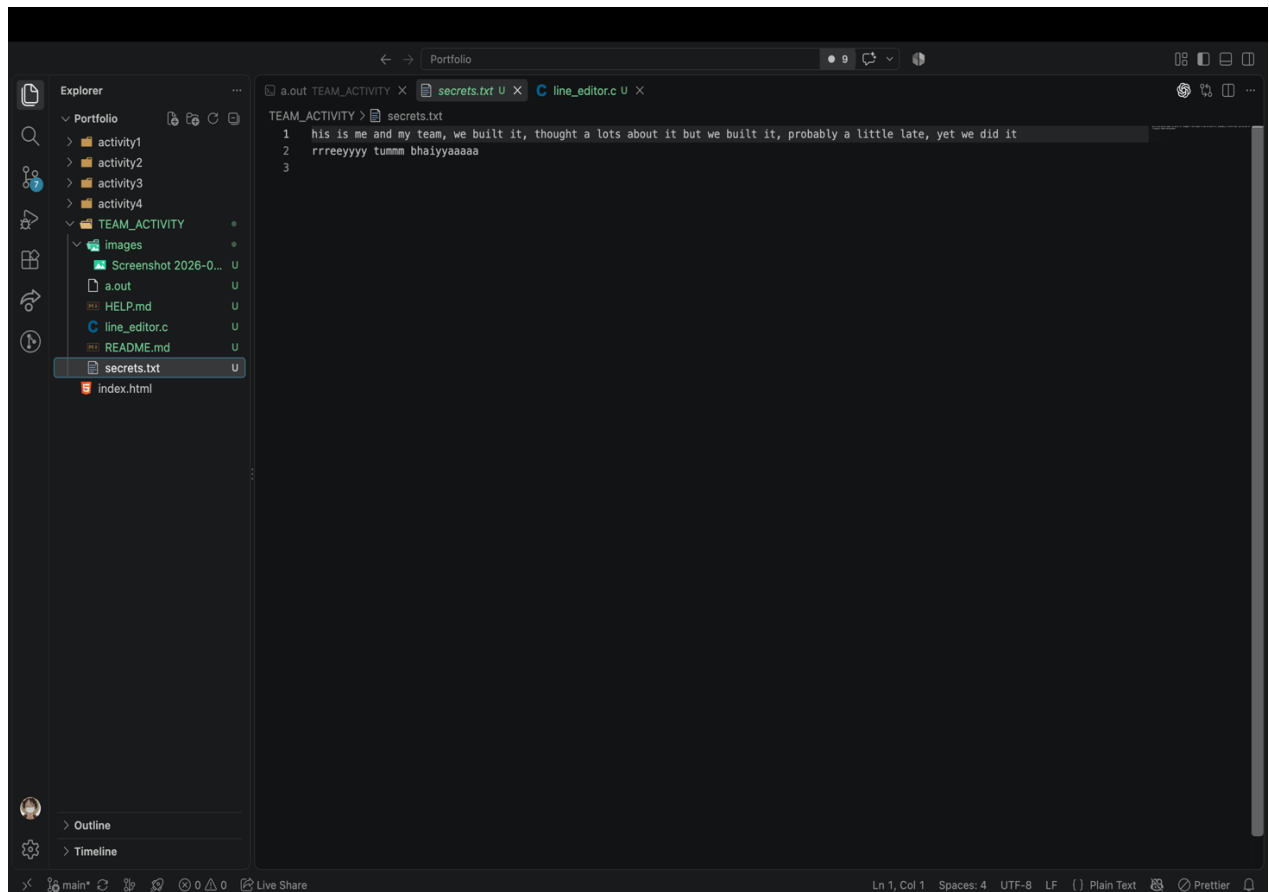
cmd> l
secrets.txt
Document loaded from secrets.txt successfully.

cmd> d
2
Error: Invalid line number.

cmd> i
rrreeyyyyy tummm bhaiyyaaaaa
Line inserted successfully.

cmd>
```

4.VERIFICATION INTO THE FILE FOR TEXT INSERT, DELETE AND UPDATE



```
TEAM_ACTIVITY > secrets.txt
1 his is me and my team, we built it, thought a lots about it but we built it, probably a little late, yet we did it
2 rrreeyyyyy tummm bhaiyyaaaaa
3
```

5.GITHUB REPRESENTATION OF THE FILES MADE(C CODE AND MD FILES)

Files

main

Go to file

TEAM_ACTIVITY

images

HELP.md

README.md

a.out

line_editor.c

secrets.txt

activity1

activity2

activity3

activity4

index.html

monkeydraco commit for team activity

a52ce90 · 2 minutes ago History

Name	Last commit message	Last commit date
..		
images	commit for team activity	2 minutes ago
HELP.md	commit for team activity	2 minutes ago
README.md	commit for team activity	2 minutes ago
a.out	commit for team activity	2 minutes ago
line_editor.c	commit for team activity	2 minutes ago
secrets.txt	commit for team activity	2 minutes ago

README.md

Advanced Command-Line Line Editor in C

Team Details & Functional Contributions

- TANMAYI M (R25EF285)**
 - Modules Developed: Document display handler (`display_document`), File I/O operations (`save_file` , `load_file`), and the core terminal command-line parsing loop (`main`).
- TARUN S MUDHOL (R25EF288)**
 - Modules Developed: Line insertion logic (`insert_line`) with automated index shifting, memory boundary protection, and syntax validation.
- PRAJWAL S NAIK (R25EF189)**
 - Modules Developed: Line deletion logic (`delete_line`), Undo transaction system (`undo_last_action`), and Keyword search implementation (`search_keyword`).

6.SNAPSHOT OF THE HELPME.MD FILE

Preview HELP.md

Line Editor Help Documentation

Welcome to the Advanced Line Editor! Below are the supported terminal commands and their usage syntax, including extra credit features.

Supported Commands

- p (Display)**
 - Description: Displays all current lines with their line numbers.
 - Example: `cmd> p`
- i <line_number> <text> (Insert Line)**
 - Description: Inserts a new line of text at a specific index, shifting existing lines down.
 - Example: `cmd> i 1 Hello World`
- d <line_number> (Delete Line)**
 - Description: Removes a line at a specific index, shifting subsequent lines up.
 - Example: `cmd> d 1`
- u (Undo Last Action - BONUS)**
 - Description: Reverses the most recent insert or delete operation.
 - Example: `cmd> u`
- f <keyword> (Search - BONUS)**
 - Description: Finds and reports which line numbers contain a given word or phrase.
 - Example: `cmd> f World`
- s <filename> (Save File)**
 - Description: Writes the current in-memory document to a text file.
 - Example: `cmd> s notes.txt`
- l <filename> (Load File)**
 - Description: Reads a text file into the editor memory.
 - Example: `cmd> l notes.txt`
- q (Quit)**
 - Description: Exits the line editor.
 - Example: `cmd> q`

7.SNAPSHOT OF README.MD FILE

Preview README.md

Advanced Command-Line Line Editor in C

Team Details & Functional Contributions

TANMAYI M (R25EF285)

Modules Developed: Document display handler (`display_document`), File I/O operations (`save_file`, `load_file`), and the core terminal command-line parsing loop (`main`).

TARUN S MUDHOL (R25EF288)

Modules Developed: Line insertion logic (`insert_line`) with automated index shifting, memory boundary protection, and syntax validation.

PRAJWAL S NAIK (R25EF189)

Modules Developed: Line deletion logic (`delete_line`), Undo transaction system (`undo_last_action`), and Keyword search implementation (`search_keyword`).

Architecture & Technical Design (How It Is Built)

1. Data Structure Choice: 2D Static Array

Instead of dynamic linked lists (which introduce pointer complexity and potential memory leaks under strict time constraints), this editor utilizes a fixed-size 2D character array:

```
char lines[MAX_LINES][MAX_LEN];
```

Trade-off & Justification:

Pros: Extremely fast, predictable memory layout, zero dependency on heap memory management (`malloc/free`), and highly stable for a bounded 2-hour terminal project.

Cons: Fixed capacity (`MAX_LINES = 100`, `MAX_LEN = 256`), meaning the document holds up to 100 lines. This trade-off was intentionally chosen to guarantee clean compilation and zero segmentation faults during evaluation.

2. Core Operational Logic

Insertion & Deletion Shifting: When a line is inserted at a specific index, all subsequent lines to the end of the document are shifted downward using `strcpy()` within a reverse loop. Deleting a line shifts subsequent lines upward to keep text contiguous in memory.

Command Parsing Loop: The program operates through an interactive REPL (Read-Eval-Print Loop) inside `main()`, reading string tokens via `scanf()` and dispatching control flow using standard `strcmp()` checks. Input buffer flushing (`getchar()`) prevents synchronization issues between numeric inputs and text entry.

3. Bonus Architecture (Undo & Search)

Single-Level Undo Engine: Managed via state tracking variables (`last_action`, `undo_line_num`, `undo_text`). When an insert or delete occurs, prior line state/metadata is safely cached. Triggering `u` calls the inverse operation natively without infinite recursion overhead.

Pattern Matching: The search module (`f`) scans the array linearly using standard library string routines (`strstr`), reporting matches instantly alongside line numbers.

Features Implemented

Core Features:

1. Insert a Line (`i`)

2. Delete a Line (`d`)

3. Display Document (`p`)

4. Save / Load File (`s` / `l`)

Bonus Features (Extra Credit):

1. Undo Last Action (`u`)

2. Keyword Search (`f`)